

주택 수리 데이터를 이용한 프로세스 마이닝

목차

01

프로젝트 개요

02

데이터 전처리

03

탐색적 자료 분석

04

프로세스 마이닝

05

프로세스 분석

06

프로세스 개선 방안

프로젝트 개요



분석 목표 및 분석 방법

- 분석 목표
 - 데이터의 프로세스 흐름을 다양한 프로세스 마이닝 알고리즘을 통해 파악.
 - 프로세스 내부에 존재하는 병목 현상, 반복 작업, 비효율 구간을 식별
- 분석 방법
 - 담당자 및 리소스 분석: 담당자별 처리량과 담당하는 주요 태스크를 분석하여 역할과 부하를 파악.
 - DFG 분석: 전체 프로세스 흐름을 시각화하고, 가장 빈번한 경로와 예외 경로를 파악.
- 사용 기술: pandas, numpy, pm4py, scipy

프로젝트 개요

컬럼 이름	데이터 타입	비고
caseID	int64	집 수리 요청 접수 번호
taskID	object	업무
originator	object	업무 담당자
Event type	object	업무의 시작과 끝을 표시
contact	object	집 수리 요청의 요청 채널
Repair Type	object	수리 방식
Object Key	float64	수리 대상 집 key
Repair Internally	object	내부 수리 여부
Estimated Repair Time	float64	예상 수리 시간
Repair Code	float64	수리 종류
Repair OK	object	수리 정상 종료 여부
date	object	업무 수행 일자
time	object	업무 수행 시각

데이터 소개

- 사용 데이터: 네덜란드 임대 주택 기관의 집 수리 요청 처리 프로세스 데이터
- 데이터 크기: 13263행 × 13열
- 핵심 컬럼
 - caseID: 개별 수리 프로세스를 식별하는 고유 ID
 - taskID: MakeTicket, Survey, InternRepair 등 프로세스 내의 개별 활동
 - originator: 해당 활동을 수행한 담당자 또는 시스템

데이터 전처리

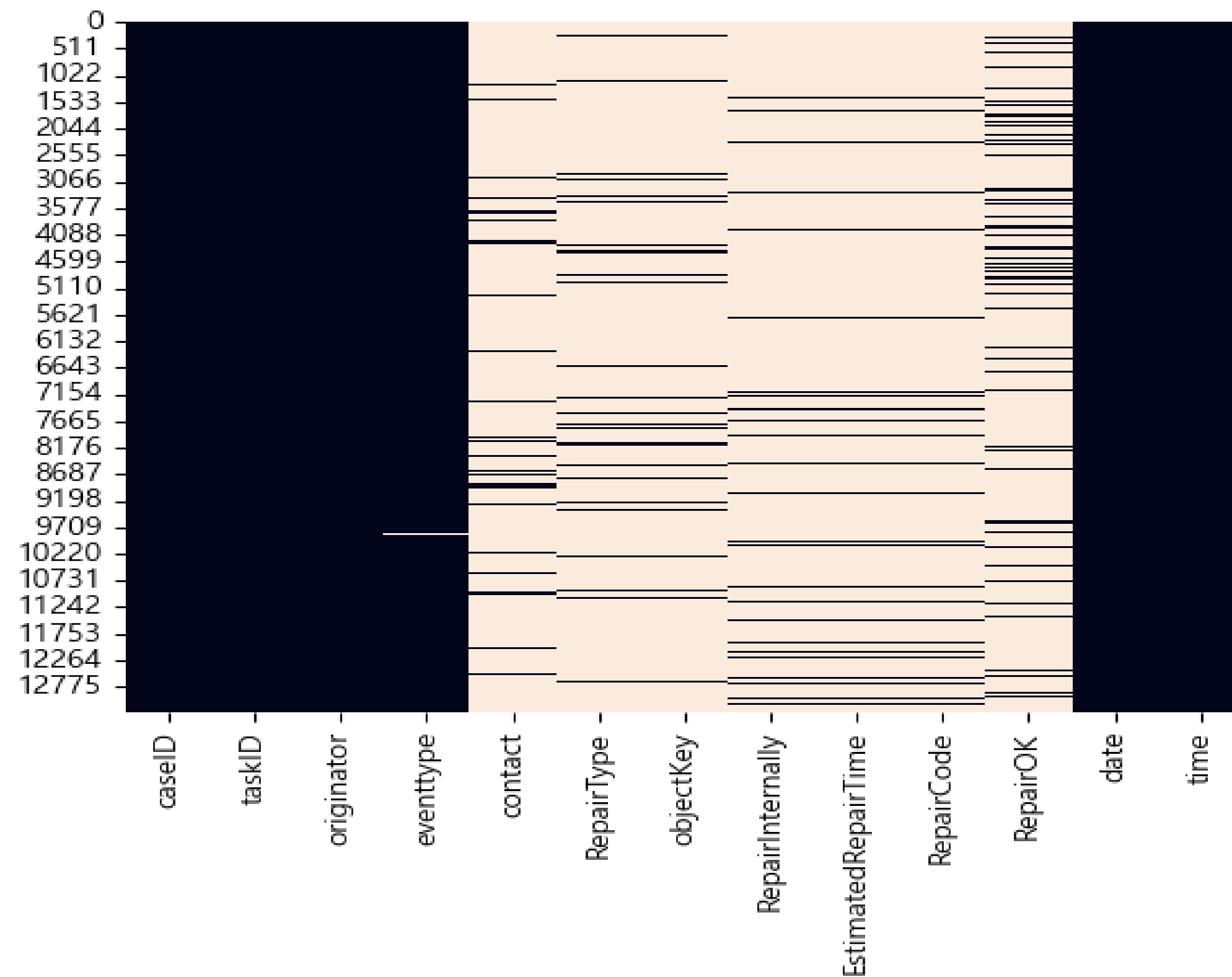
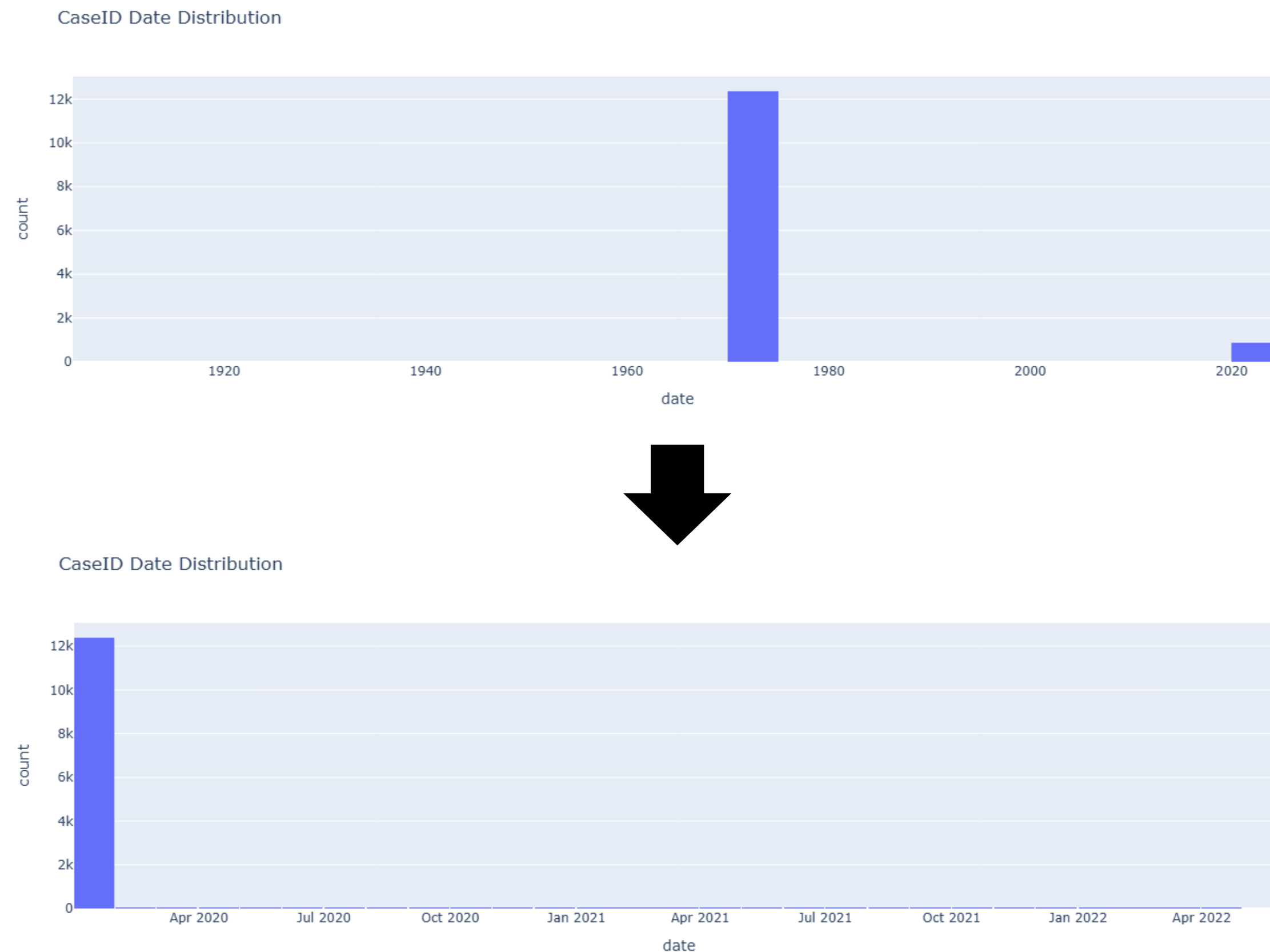


Figure 1. 원본 데이터의 결측값 히트맵

기본 전처리

- 결측값: 앞서 정의한 주요 컬럼의 결측값들은 제거.
- 중복값: 데이터 확인 결과 중복값은 존재 하지 않았음.
- 분석에 필요하지 않은 컬럼 제거: 결측이 많은 일부 컬럼들을 제거하여 프로세스 마이닝에서는 사용하지 않음. (데이터 필터링에는 사용)
- 데이터 형변환
 - date와 time 컬럼을 결합해 datetime이라는 파생 변수를 생성후 날짜형 데이터로 형변환.

데이터 전처리



시간(time, date)에 대한 문제 상황 (1)

- 문제점: 결측된 데이터를 제외한 원본 데이터에서 date 컬럼의 대부분이 1905년, 1970년 데이터로 기록되어 있음.
- 가설: 시스템 마이그레이션 또는 대규모 업데이트 과정에서 날짜 필드 초기화 발생 → 최근 일자로 전처리 필요
- 처리 방안
 - 문제 년도로 기록된 모든 이벤트를 2020년의 동일 월·일·시간으로 일괄 보정.
 - 시간 정보가 결측된 데이터는 알 수 없었기 때문에 제거.

Figure 2. 전처리 이전-이후 데이터의 시간 분포 히스토그램

데이터 전처리

taskID	Originator	Event Type	Datetime
FirstContact	Dian	Complete	2020-01-03
MakeTicket	Dian	Start	2020-01-03
ArrangeSurvey	Dian	Start	2020-01-03
ArrangeSurvey	Dian	Complete	2020-01-03
InformClientSurvey	System	Complete	2020-01-03
Survey	Cindy	Start	2020-01-06
Survey	Cindy	Complete	2020-01-06
...			
ReadyInformClient	System	Complete	2020-01-06
TicketReady	System	Complete	2020-01-06
MakeTicket	Dian	Complete	2022-02-17

Table 1. 해당 상황을 가진 71번 Case ID 프로세스 데이터

시간(time, datetime)에 대한 문제 상황 (2)

- 문제점: datetime 전처리 이후 399개의 Case에서 일부 Task가 2021년 또는 2022년으로 기록됨
- 가설: time 값은 정상이나 date 필드만 잘못 덮어 씌워졌을 것이다. → Task의 시작과 끝 쌍이 맞도록 전처리
- 처리 방안
 - Start와 Complete가 동일하지 않은 날짜일 경우 해당 이벤트의 날짜 쌍을 논리에 맞게 보정.
 - 시간은 그대로 유지.
 - 동일 `caseID` 내에서 major date 기준으로 벗어난 이벤트 탐지 후 ±30일 이상 벗어난 경우 날짜를 major date로 보정

탐색적 자료 분석

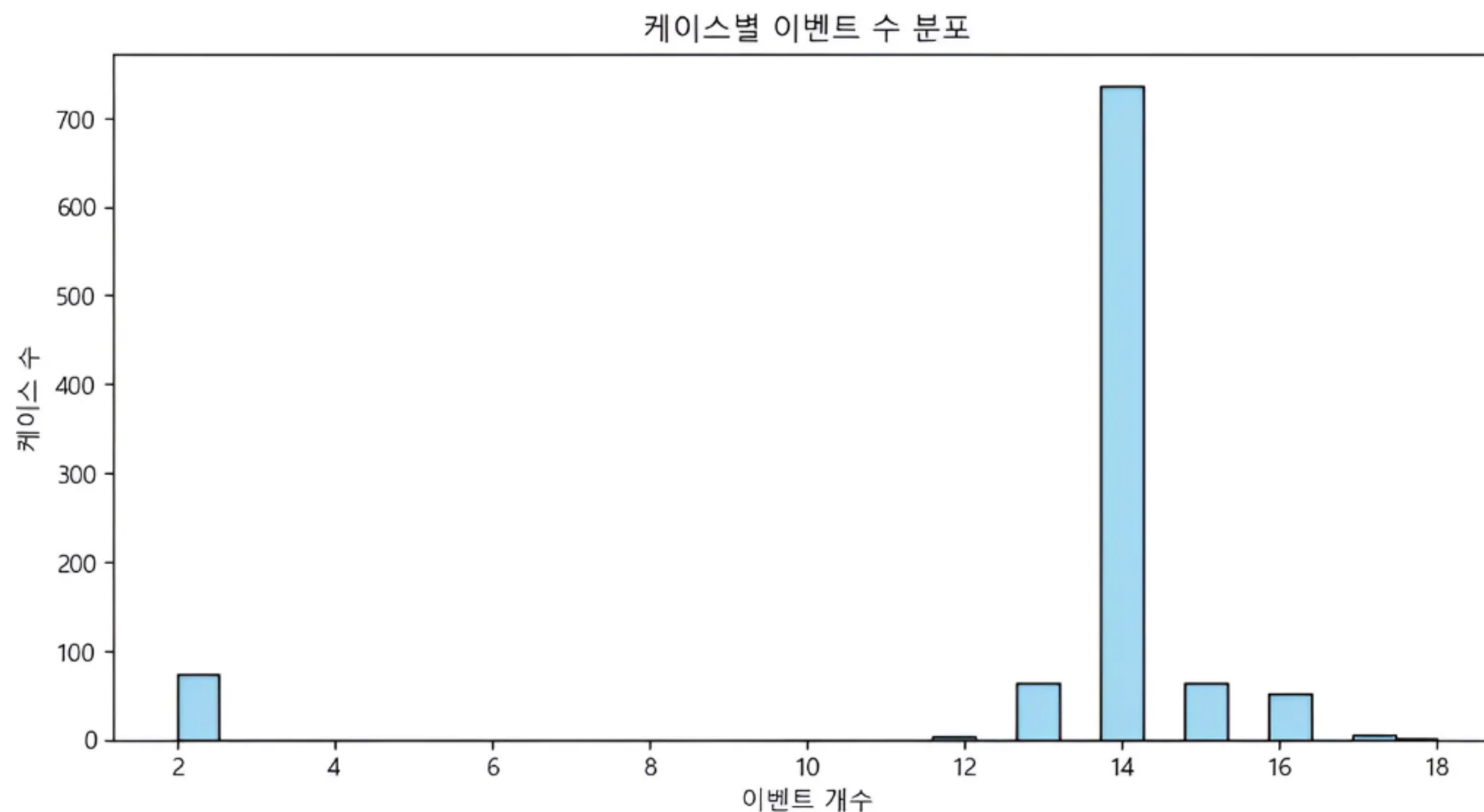


Figure 3. 케이스별 이벤트 수 분포 히스토그램

케이스 별 이벤트 수 분포

- 다수 케이스 = 14개 이벤트 → 표준화된 프로세스가 자리 잡아 있을 가능성 높음.
- 14개 미만·초과 케이스 존재 → 단계 누락/반복·재처리로 인한 변형 및 병목 징후 의심.
- 이벤트가 2개인 소수 케이스 → 미완료·중단 건이거나 로그 결측/오입력 가능성. 별도 정제 및 원인 추적 필요. → 이후 확인 결과 InformClientWrongPlace라는 이벤트로만 이루어진 Task.

탐색적 자료 분석

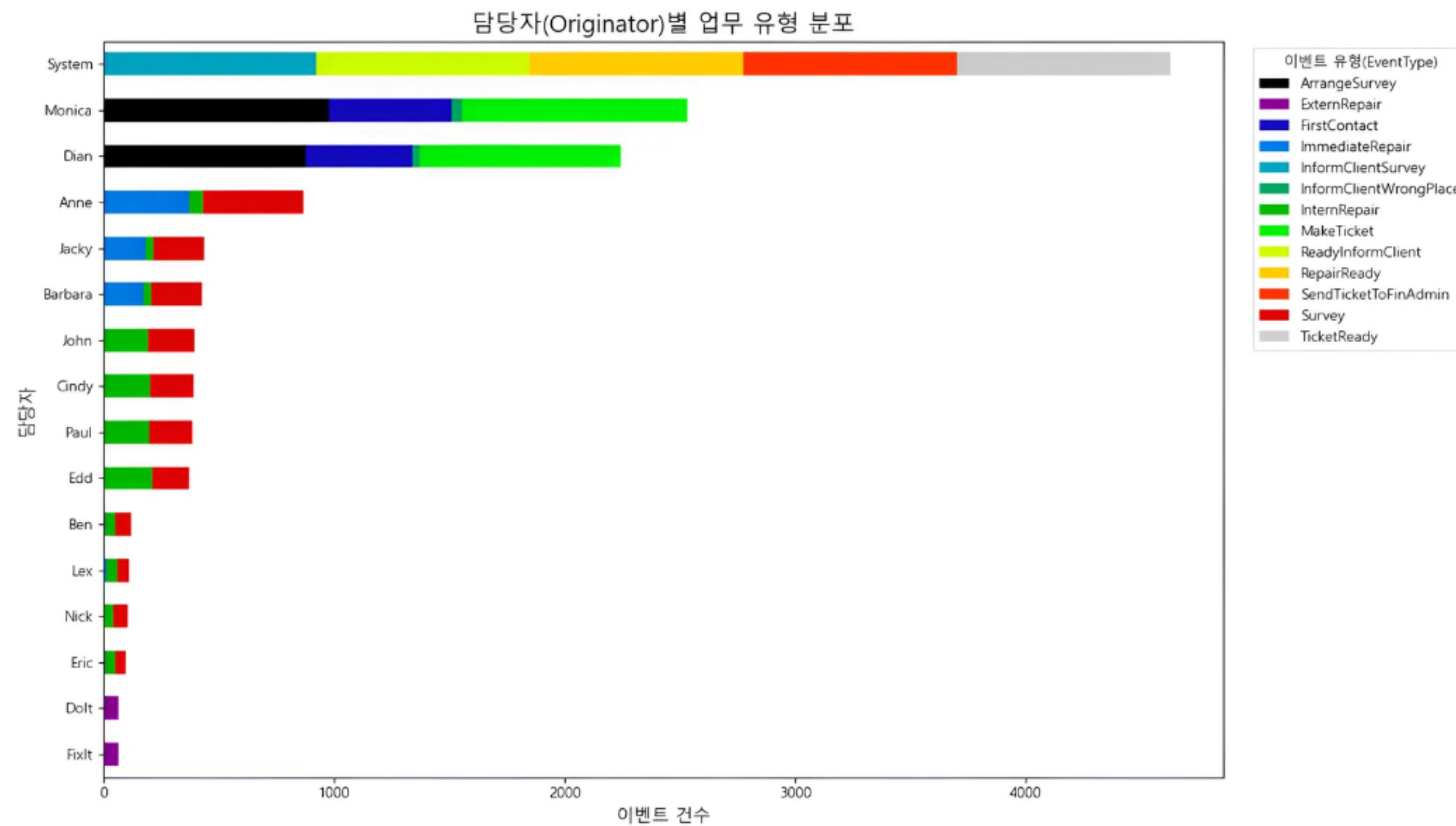


Figure 4. 담당자 별 업무 유형 건수 막대 그래프

담당자별 업무 유형 분포

- 자동화 비중이 가장 큼: System이 전체 이벤트의 최다 비중을 차지함. 주로 후행 및 알림성 프로세스를 담당하며 자동 처리됨
- 편중된 업무: Monica, Dian은 주로 고객과의 소통을 담당하는 인력. 그러나 타 인원 대비 조사 인력은 업무 처리량이 압도적이기 때문에 해당 프로세스에서 병목 또는 지연 가능성을 확인
- 수리를 담당하는 인력: Anne, Jacky, Barbara는 Immediate Repair에 특화된 인력. 이외의 인력들은 수리가 필요한 곳을 조사하거나 외부 업체의 인력임.

탐색적 자료 분석

케이스 별 이벤트 타임라인

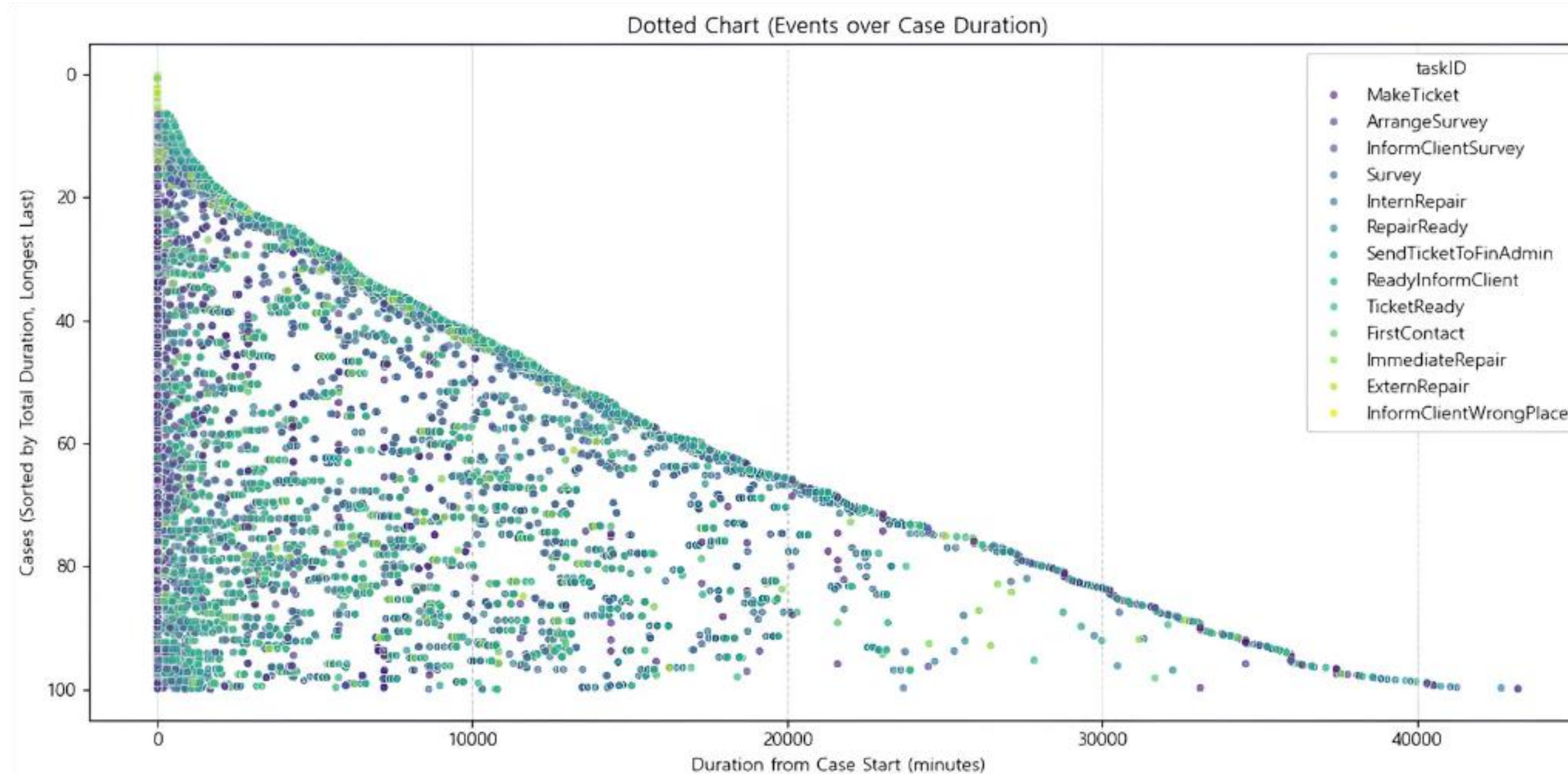
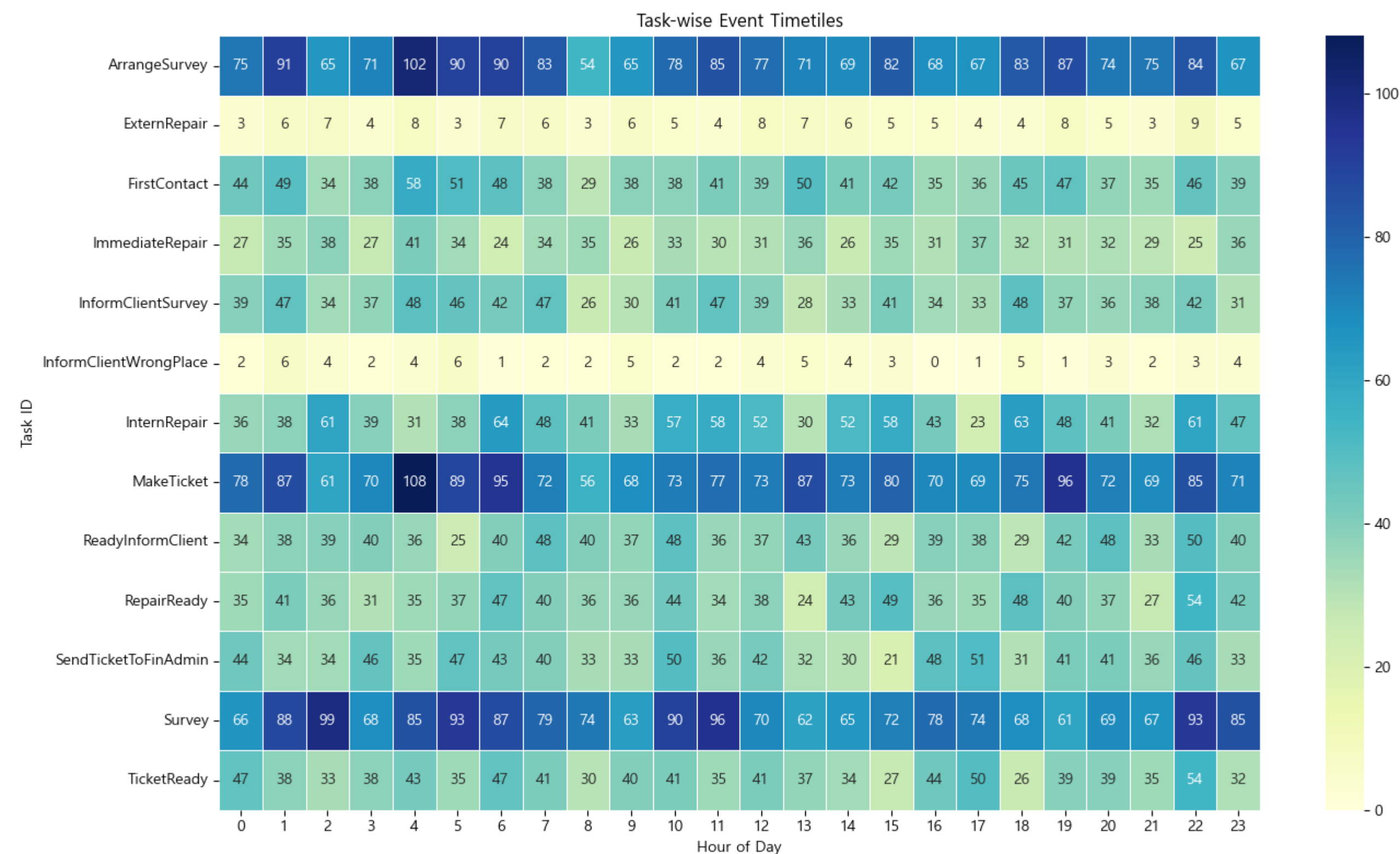


Figure 5. 케이스별 이벤트 타임라인 Dotted Chart

- 강한 롱테일: 대부분의 Case는 시작후 짧은 시간 내 종료되고, 일부 소수 케이스가 최대 3일까지 이어지며 전체 리드타임에 편향된 결과를 불러옴
- 초기 이벤트 밀집: FirstContact, MakeTicket, ArrangeSurvey 등 접수와 배정 단계 이벤트는 대부분 즉시 끝나는 경향 → 시작 직후 활동이 집중
- 지속 시간은 대기 간격에 영향을 줌: 장기로 늘어진 케이스일 수록 점들 사이 간격이 넓음. 특히 지연 케이스에서는 Immediate Repair, Extern Repair가 반복 발생 → 병목 가능성

탐색적 자료 분석



Task 별 시간대 패턴

- 근무 시간대는 사람 업무 중심: InternRepair / InformClientSurvey / ReadyInformClient는 09-18시에 고르게 증가 → 인력 배치가 실제 처리 수요와 대체로 정합.
- 종료·인계성 이벤트의 오후 상승: RepairReady / TicketReady / SendTicketToFinAdmin이 오후~저녁(17-22시)에 완만히 상승 → 작업 마감·인계가 일과 후반에 집중.
- 예외 흐름은 희소함: ExternRepair / InformClientWrongPlace는 전 시간대에 낮고 평탄 → 예외 케이스 전용 모니터링을 별도로 설정하는 것이 효율적.

Figure 6. Task 별 시간대 패턴 Timetiles 히트맵

프로세스 마이닝

CaseID	TaskID	Start Time	Complete Time
1	FirstContact	2020-01-02 8:08	2020-01-02 8:08
1	MakeTicket	2020-01-02 8:08	2020-01-02 8:11
1	ArrangeSurvey	2020-01-02 8:11	2020-01-02 8:16
1	InformClientSurvey	2020-01-02 8:16	2020-01-02 8:16
1	Survey	2020-01-11 21:33	2020-01-11 21:33
1	InternRepair	2020-01-17 4:36	2020-01-17 8:12
1	RepairReady	2020-01-17 8:12	2020-01-17 8:12
1	SendTicket	2020-01-17 14:03	2020-01-17 14:03
1	ReadyInformClient	2020-01-17 15:44	2020-01-17 15:44
1	TicketReady	2020-01-17 15:44	2020-01-17 15:44

Table 3. 가공된 1번 Case 데이터 예시

프로세스 분석을 위한 데이터 가공

- 프로세스 로그 정규화: 원천 데이터를 표준 스키마(caseID, taskID, originator, datetime)로 통일해 케이스 단위 시퀀스로 재구성.
- 시퀀스 품질 정리: 케이스 내 시간 기준 정렬 후 기본 결측·중복 점검(필요 시 보정)으로 분석 가능한 이벤트 열로 정리.
- Start-Complete 파생 규칙: 인력 업무는 기존 Start/Complete를 사용하고, System 단일 이벤트는 즉시 완료(lead time=0) 가정으로 Start-Complete 쌍을 생성.



- Figure 7. DFG Plot

프로세스 마이닝

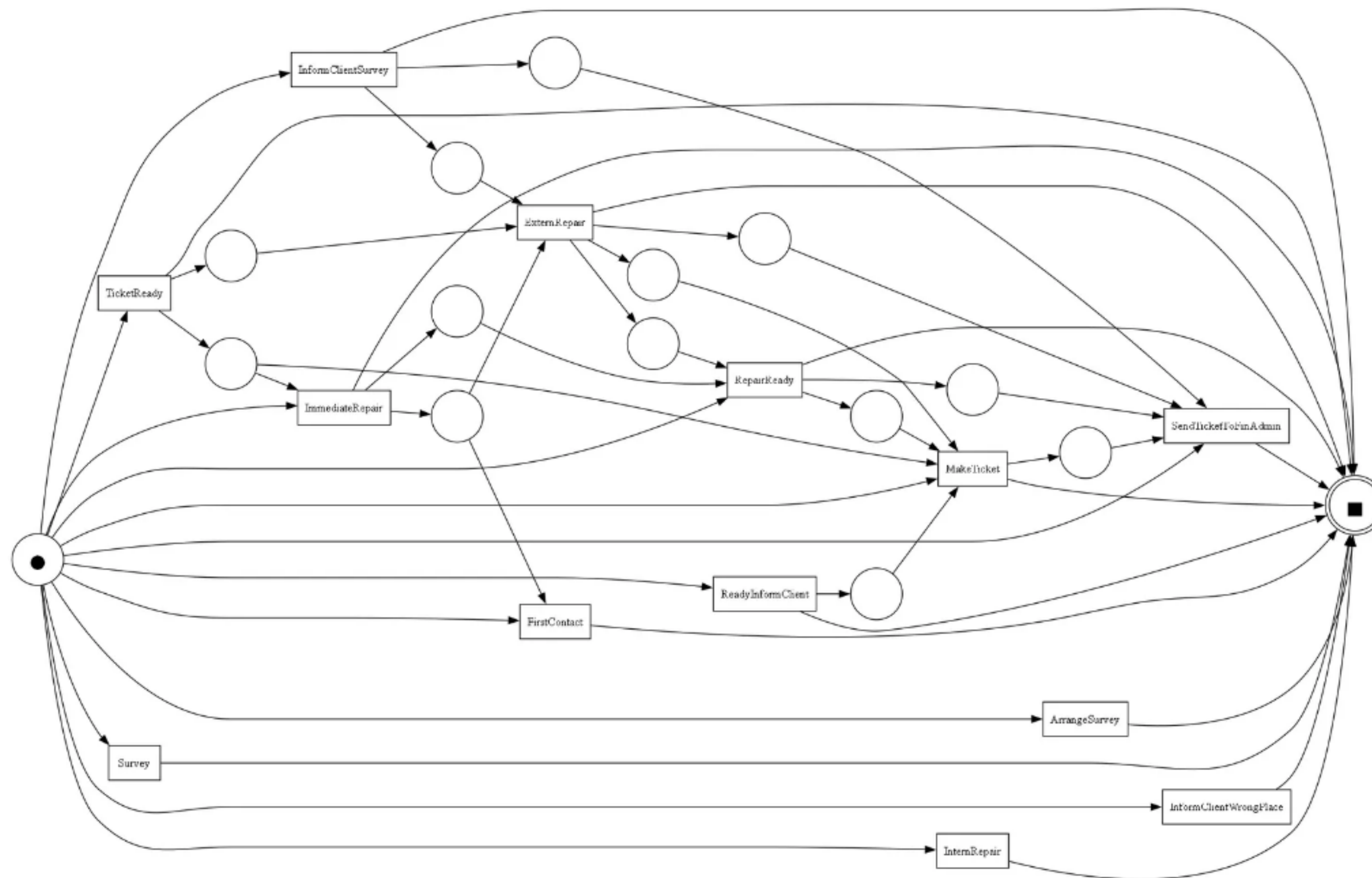


Figure 8. 알파 마이너 프로세스 흐름도

프로세스 마이닝 결과 - 알파 마이너

- 분석 결과: 알파 마이너를 통해 도출한 프로세스 모델은 실제 업무 흐름 프로세스를 반영하지 못하는 스파게티 형태를 띠
- 주요 특징
 - 작업 간 관계를 명확히 표현하지만, 병렬 처리 및 예외 경로의 표현은 부족함.
 - Survey, ArrangeSurvey, InformClientSurvey 등의 핵심 활동 간 전이는 비교적 단순하게 연결됨.

프로세스 마이닝

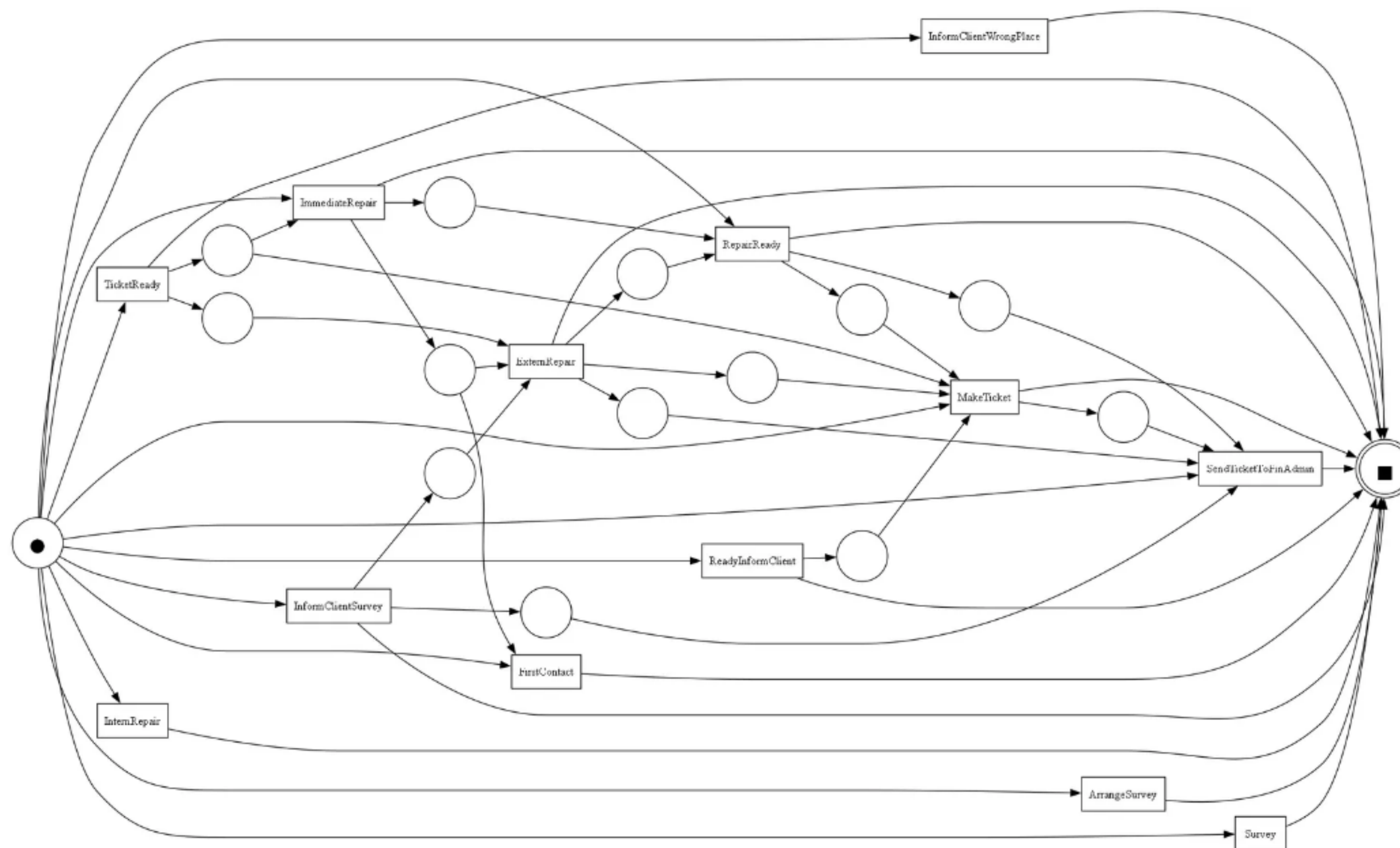
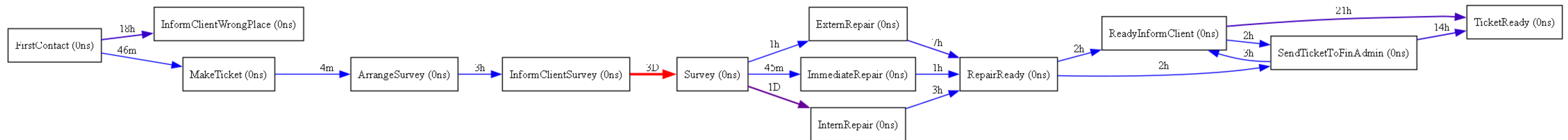


Figure 9. 휴리스틱 마이너 프로세스 흐름도

프로세스 마이닝 결과 – 휴리스틱 마이너

- 분석 결과: 빈도 기반의 패턴을 반영한 휴리스틱 마이너는 실제 업무 흐름을 더 현실적으로 표현하는 장점을 지니지만, 현재 프로세스에서도 알파 마이너와 비슷한 스파게티의 형태를 띠
- 주요 특징
 - ExternRepair, InternRepair, ImmediateRepair 등 다중 경로가 병렬로 유지되어, 실제 운영 환경의 복잡성을 반영.

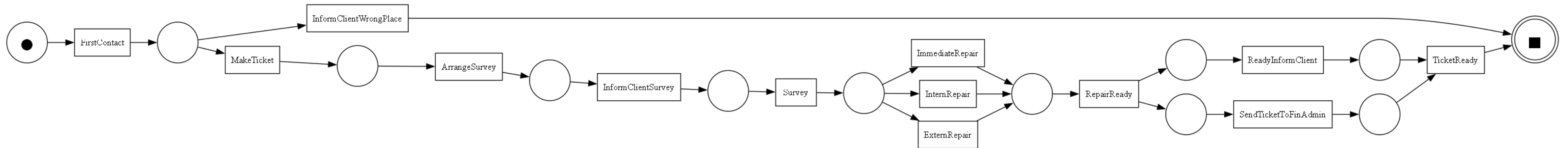
프로세스 마이닝



프로세스 마이닝 결과 - DFG (Variants 50%)

- 분석 결과: Variants=0.5를 적용한 DFG는 전체 DFG 모델의 스파게티 구조를 단순화하여, 주요 경로 중심의 핵심 프로세스 모델을 도출함.
- 주요 특징
 - FirstContact → MakeTicket → Survey → InformClientSurvey → Repair 단계 → RepairReady → TicketReady로 이어지는 프로세스가 핵심 프로세스로 추정됨.
 - ReadyInformClient, SendTicketToFinAdmin에서 반복되는 프로세스가 존재.
 - InformClientSurvey → Survey에서 병목을 의심. (3일 소요)

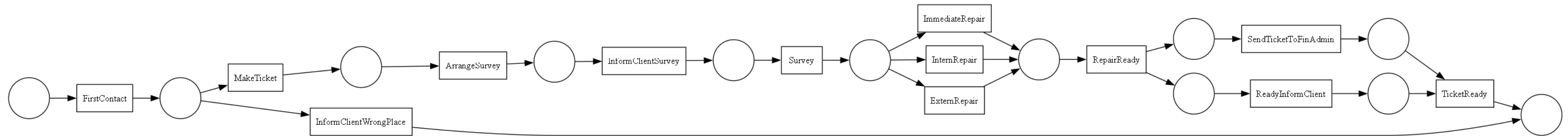
프로세스 마이닝



프로세스 마이닝 결과 – 알파 마이너 (Variants 50%)

- 분석 결과: Variants=0.5 조건으로 필터링한 알파 마이너 모델은 전체 알파 마이너보다 더욱 간결하며 핵심 경로 중심으로 표현 됨.
- 주요 특징
 - DFG와 마찬가지로 FirstContact → MakeTicket → Survey → InformClientSurvey → Repair 단계 → RepairReady → TicketReady로 이어지는 프로세스가 집 수리의 핵심 프로세스임을 확정 할 수 있었음.

프로세스 마이닝



프로세스 마이닝 결과 – 휴리스틱 마이너 (Variants 50%)

- 분석 결과: 상위 50%의 프로세스를 필터링 한 결과 DFG, 알파마이너, 휴리스틱 마이너가 서로 같은 프로세스의 형태를 띄었음.
- 주요 특징
 - 앞선 프로세스들과 마찬가지로 FirstContact → MakeTicket → Survey → InformClientSurvey → Repair 단계 → RepairReady → TicketReady로 이어지는 프로세스가 집 수리의 핵심 프로세스임을 확정 할 수 있었음.

프로세스 분석

- 수리 프로세스 진행 방식: 프로세스 마이닝 분석 결과를 통해 도출된 프로세스는 크게 4단계의 흐름을 따르는 것으로 확인할 수 있었습니다.

1. 접수 및 티켓 생성: 고객의 최초 수리 요청(FirstContact)부터 현장 점검을 준비하는 단계.

- 프로세스 흐름: FirstContact → MakeTicket → ArrangeSurvey → InformClientSurvey

2. 현장 점검: 배정된 담당자(Originator)가 현장을 방문하여 문제점을 진단하는 핵심 단계. 이 단계의 완료 후, 수리 방향을 결정.

- 프로세스 흐름: ArrangeSurvey → InformClientSurvey → Survey

3. 수리 단계: 진단 결과에 따라 아래 세 가지 유형 중 하나로 분기하여 수리를 진행.

- InternRepair: 내부 수리팀이 직접 처리.
- ExternRepair: 외부 전문 업체(FixIt, DoIt)로 수리를 이관.
- ImmediateRepair: 현장에서 즉시 처리가 가능한 경우, 짧은 시간 내에 수리를 처리.

4. 수리 완료 및 티켓 종료: 수리가 완료된 후, 고객에게 통보하고 내부적으로 재무 처리 등을 거쳐 티켓을 최종 종결하는 단계.

- 프로세스 흐름: RepairReady → ReadyInformClient / SendTicketToFinAdmin → TicketReady

전체 Process

FirstContact → MakeTicket → ArrangeSurvey → InformClientSurvey → Survey → RepairReady →
ReadyInformClient → SendTicketToFinAdmin → TicketReady

프로세스 분석

- 현재 프로세스의 주요 문제점: 프로세스 전반에 걸쳐 크게 병목 현상과 반복 작업이라는 두 가지 문제점을 발견하였습니다.

1. 병목 현상

I. Survey 단계의 심각한 대기 시간

- 분석 결과: DFG 분석에 따르면 InformClientSurvey → Survey 구간에서 평균 약 3일의 대기 시간이 발생하며, 프로세스 전반에서 가장 큰 병목 지점으로 확인됨.
- 원인
 - 인력 부족: Survey 조율을 담당하는 인원이 Monica, Dian 두 명으로 한정되어 있음.
 - 수동 스케줄링: 고객 일정 조율과 담당자 배정 과정이 자동화되지 않아 지연 누적.

2. 반복 작업 및 비효율 작업

I. ReadyInformClient ↔ SendTicketToFinAdmin 병렬 실행

- 분석 결과: 수리 완료 후 티켓을 종결하는 과정에서 두 태스크가 명확한 순서 없이 병렬 실행되며, 최종적으로 TicketReady로 수렴.
- 문제점
 - 의존 관계 불명확: 두 태스크 간 순서가 정의되지 않아 중복 작업 또는 재처리 발생 가능성.
 - 자동화 비효율: 해당 흐름이 시스템에서 자동화되어 있으나, 구조적 비효율로 인해 미세한 지연이 지속적으로 축적될 가능성 존재.

프로세스 분석

- 케이스 소요 시간에 영향을 미치는 데이터 요소는 ExternRepair 라고 판단하였습니다.

1. 숨겨진 프로세스로 인한 리드타임 증가

- 문제점: RepairInternally 컬럼이 True임에도 불구하고 몇몇 태스크는 ExternRepair를 수행하였음. 이 ExternRepair는 시작 시간에 대한 데이터만 존재하고 완료 시간은 따로 기록되어 있지 않음. → 실제 활동이 진행 중임에도 불구하고 데이터상으로는 긴 대기 시간으로 인식되는 문제가 발생.
- 실험 가설: InternRepair 또는 ImmediatRepair 도중 수리 지연 또는 실패로 인한 ExternRepair로 전환하는 구간에서 추가적인 리드타임 발생한다고 가정.

2. 케이스 소요 시간 분석

- 집단 정의: 케이스를 세 그룹으로 분류하여 분산분석을 수행
 - I. 내부 업체 수리 (InternRepair)
 - II. 외부 업체 수리 (ExternRepair)
 - III. 수리 중 이관 (InternRepair or ImmediatRepair → ExternRepair)
- 정규성 검정: 데이터에 로그 변환을 수행한 뒤에도 세 집단의 p-value = 0.000 < 0.05으로 모두 정규성을 만족하지 않았음.
- 등분산성 검정: 마찬가지로 데이터에 대해 등분산 검정 수행 결과 p-value = 0.019 < 0.05로 등분산 가정도 만족하지 않아 비모수적 분산분석을 대신 수행
- Kruskal-Wallis 검정
 - H0: 세 집단의 소요 시간 분포는 동일하며, 따라서 중앙값의 차이가 없다.
 - H1: 세 집단 중 적어도 한 집단은 다른 집단과 중앙값이 다르다.
 - 검정 결과: p-value = 0.0075 < 0.05 → 세 집단 간 소요 시간 차이가 통계적으로 유의함.
 - 사후 검정: 내부업체 수리 vs 내부→외부 이관의 p-value = 0.0075, 내부업체 수리 vs 외부업체 수리의 p-value = 0.0282, 수리중 이관 vs 외부업체 수리의 p-value = 0.0185라는 결과로 세 집단 모두 소요시간에 대해 유의미한 차이를 보였음

프로세스 개선 방안

프로세스 개선 방향 및 기대 효과

개선 영역	문제점	개선 아이디어	기대 효과
Survey 단계	인력 부족, 대기시간 증가	- Survey 인원 증원 및 대기 시간 단축 - 우선순위 기반 자동 스케줄러 도입	Survey 대기시간 단축 및 리드타임 안정화
수리 중 이관	리드 타임 증가	- Survey 직후 ExternRepair 여부 조기 판단 - 업체 자동 배정 시스템 구축	ExternRepair 호출 지연 최소화 및 데이터 정합성 향상
System 태스크 단순화	ReadyInformClient ↔ SendTicketToFinAdmin 경로 중복	ExternRepair 과정 중 숨은 프로세스 가시화	숨은 병목 구간 가시화 및 관리 효율 극대화
ExternRepair 로그 품질	완료 이벤트 및 세부 상태 누락	두 태스크의 프로세스 표준화 및 자동화 구조 개선	프로세스 단순화 및 처리 속도 향상